```
# Top Selling Vendors
# Show the top 5 vendors who sold the highest total quantity of products across all market dates.
# Show vendor id, vendor name and total quantity sold.

select * from customer_purchases limit 10;
select * from vendor;

select
        v.vendor_name,
        round(sum(cp.quantity), 2) as total_quantity_sold
from vendor v
inner join customer_purchases cp
on v.vendor_id = cp.vendor_id
group by v.vendor_name
order by 2 DESC
limit 5;
```

```
# Booths That Were Never Assigned
# Find booths from the booth table that were never assigned to any vendor.
# show booth_number, booth_price_level, booth_description, booth_type

select * from booth;
select * from vendor_booth_assignments;

select b.* from
booth b
left join vendor_booth_assignments vba
on b.booth_number = vba.booth_number
where vba.booth_number is null
order by booth_number;
```

```
# Repeat Customers
# List all customers who have made purchases on at least 3 different market dates.
# Show their first name, last name, zip code, and count of distinct market dates.

select * from customer limit 10;
select * from customer_purchases limit 10;

select
        c.customer_first_name,
        c.customer_last_name,
        c.customer_zip,
        count(cp.market_date) as count_of_distinct_market_dt
from customer c
inner join customer_purchases cp
on c.customer_id = cp.customer_id
group by 1, 2, 3
having count_of_distinct_market_dt >= 3
order by count_of_distinct_market_dt desc;




# Product Price Variations
# For each product, show the minimum and maximum price charged to customers
(cost_to_customer_per_qty).
# Sort by the difference in price (highest difference first).
# show the product_id, product_name, min_price, max_price, price_diff

select * from product;
select * from customer_purchases;

select
        p.product_id,
        p.product_name,
        min(cp.cost_to_customer_per_qty) as min_price,
        max(cp.cost_to_customer_per_qty) as max_price,
```

```
        max(cp.cost_to_customer_per_qty) - min(cp.cost_to_customer_per_qty) as
price_diff
from product p
inner join customer_purchases cp
on p.product_id = cp.product_id
group by 1, 2
order by 5 desc;
```

```
# Understocked Products
# List all products whose inventory (in vendor_inventory) has ever gone below 10 units.
# Include the product name, market date, vendor name, and quantity.

select * from vendor_inventory;
select * from vendor;
select * from product;

select
        p.product_name,
        vi.market_date,
        v.vendor_name,
        vi.quantity
from vendor v
inner join vendor_inventory vi
on v.vendor_id = vi.vendor_id
inner join product p
on vi.product_id = p.product_id
where vi.quantity < 10;
```

```
# Booth Assignment Changes
```

```
# Find vendors who have changed booths across different market dates.
# Show vendor id, vendor name, number of unique booths they've been assigned to,
and total market appearances.

select * from vendor;
select * from vendor_booth_assignments;

select
	v.vendor_name,
	v.vendor_id,
	count(distinct vba.booth_number) as unique_booths_assigned,
	count(distinct vba.market_date) as total_market_appearance
from vendor v
inner join vendor_booth_assignments vba
on v.vendor_id = vba.vendor_id
group by v.vendor_id, v.vendor_name
having unique_booths_assigned > 1;
```

```
# Busiest Market Days
# Which 3 market dates had the highest number of total transactions (rows in
customer_purchases)?
# Show market_date, market_day, market_season, market_rain_flag,
market_snow_flag,
# highest_num_of_tot_transactions

select * from market_date_info;
select * from customer_purchases;

select
	mdi.market_date,
	mdi.market_day,
	mdi.market_season,
	mdi.market_rain_flag,
	mdi.market_snow_flag,
	count(*) as highest_num_of_tot_transactions
```

```
from market_date_info mdi
inner join customer_purchases cp
on mdi.market_date = cp.market_date
group by mdi.market_date
order by 8 desc
LIMIT 5;
```

```
# Average Spend per Customer per Visit
# show customer_id, average_amount_spent_per_visit

select * from customer_purchases;

WITH customer_sum_spend_per_visit AS (
    SELECT
        customer_id,
        market_date,
        SUM(quantity * cost_to_customer_per_qty) AS daily_total
    FROM customer_purchases
    GROUP BY customer_id, market_date
)
SELECT
    customer_id,
    AVG(daily_total) AS avg_spend_per_visit
FROM customer_sum_spend_per_visit
GROUP BY customer_id;
```

```
# First Purchase per Customer
# Find the first-ever purchase made by each customer.
```

```
select customer_id, market_date, quantity, cost_to_customer_per_qty from
customer_purchases order by customer_id, market_date;

WITH ranked_purchases AS (
  SELECT
    customer_id,
    market_date,
    product_id,
    ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY market_date) AS
row_num
  FROM customer_purchases
)
SELECT *
FROM ranked_purchases
WHERE row_num = 1;
```

```
# Top 3 Booths by Usage Count
# Count how often each booth has been assigned and rank the most used booths.

select * from vendor_booth_assignments;

with vendor_usage as (
        select
                booth_number,
                count(market_date) as usage_count
        from vendor_booth_assignments
        group by booth_number
),
ranked_vendor_usage as (
        select
                booth_number,
                usage_count,
                dense_rank() over (order by usage_count desc) as booth_rank
        from vendor_usage
```

```
)
select * from ranked_vendor_usage where booth_rank <= 3;
```

```
# Detect Price Jumps for a Product
# Find when the price for a product changed from one market date to the next.
# Show product_id, market_date, cost_to_customer_per_qty, next_price, next_date

select * from customer_purchases;


SELECT
    product_id,
    market_date,
    cost_to_customer_per_qty,
    LEAD(cost_to_customer_per_qty) OVER (PARTITION BY product_id ORDER BY
market_date) AS next_price,
    LEAD(market_date) OVER (PARTITION BY product_id ORDER BY market_date) AS
next_date
FROM customer_purchases;
```

```
# Compare Current and Previous Day’s Inventory for Vendors
# Find changes in quantity held by vendors day-to-day.
# show vendor_id, product_id, market_date, quantity, prev_quantity

select * from vendor_inventory;

SELECT
    vendor_id,
```

```
    product_id,
    market_date,
    quantity,
    LAG(quantity) OVER (PARTITION BY vendor_id, product_id ORDER BY
market_date) AS prev_quantity
FROM vendor_inventory;
```

```
# Top Selling Products per Category
# For each product category, rank products based on total quantity sold
# Show: category_name, product_name, total_quantity, rank_in_category

select * from customer_purchases;
select * from product;
select * from product_category;

SELECT
    pc.product_category_name,
    p.product_name,
    SUM(cp.quantity) AS total_quantity,
    DENSE_RANK() OVER (PARTITION BY pc.product_category_name ORDER BY
SUM(cp.quantity) DESC) AS rank_in_category
FROM customer_purchases cp
JOIN product p ON cp.product_id = p.product_id
JOIN product_category pc ON p.product_category_id = pc.product_category_id
GROUP BY pc.product_category_name, p.product_name;
```